

smallpond

Deep Dive into DeepSeek — Chapter 31

Yin Yang

Foundation Books Project

The one rule of this chapter

A logical plan is not a dataset. Only the **first successful attempt's attached output** may satisfy a downstream dependency.

- A dataframe program can be syntactically complete while its output directory is operationally unusable — a missing shuffle producer, a partial retry, two plausible speculative copies.
- `smallpond` is a lazy dataframe and task-execution layer: it turns that program into a dependency-complete, unambiguous published dataset.
- The running example, carried all chapter: a prices dataframe — `read_parquet`, `hash_repartition(3, ...)`, a partition-local SQL aggregate, `write_parquet`.

The one rule of this chapter

A logical plan is not a dataset. Only the **first successful attempt's attached output** may satisfy a downstream dependency.

- A dataframe program can be syntactically complete while its output directory is operationally unusable — a missing shuffle producer, a partial retry, two plausible speculative copies.
- smallpond is a lazy dataframe and task-execution layer: it turns that program into a dependency-complete, unambiguous published dataset.
- The running example, carried all chapter: a prices dataframe — `read_parquet`, `hash_repartition(3, ...)`, a partition-local SQL aggregate, `write_parquet`.

Goal: follow one dataframe expression through the logical plan, planner, scheduler, storage backend, and the public GraySort row — and see exactly which layer owns which guarantee.

1. From dataframe expression to published dataset

1. From dataframe expression to published dataset
2. System boundary and dataset contract

1. From dataframe expression to published dataset
2. System boundary and dataset contract
3. Logical plans, repartition, and shuffle

1. From dataframe expression to published dataset
2. System boundary and dataset contract
3. Logical plans, repartition, and shuffle
4. Execution, resource admission, and failure

1. From dataframe expression to published dataset
2. System boundary and dataset contract
3. Logical plans, repartition, and shuffle
4. Execution, resource admission, and failure
5. Storage backend, 3FS boundary, and GraySort evidence

From dataframe expression to published dataset

Five questions the chapter answers

dataframe expr $\longrightarrow$ logical plan $\longrightarrow$ task groups $\longrightarrow$ successful attempt $\longrightarrow$ published sink

1. Which calls describe a plan, and which call demands execution?

Five questions the chapter answers

dataframe expr $\longrightarrow$ logical plan $\longrightarrow$ task groups $\longrightarrow$ successful attempt $\longrightarrow$ published sink

1. Which calls describe a plan, and which call demands execution?
2. Which producer files make one destination partition complete?

Five questions the chapter answers

dataframe expr $\longrightarrow$ logical plan $\longrightarrow$ task groups $\longrightarrow$ successful attempt $\longrightarrow$ published sink

1. Which calls describe a plan, and which call demands execution?
2. Which producer files make one destination partition complete?
3. Which resources admit a task to an executor?

Five questions the chapter answers

dataframe expr $\longrightarrow$ logical plan $\longrightarrow$ task groups $\longrightarrow$ successful attempt $\longrightarrow$ published sink

1. Which calls describe a plan, and which call demands execution?
2. Which producer files make one destination partition complete?
3. Which resources admit a task to an executor?
4. Which retry or speculative attempt supplies the consumed dataset?

Five questions the chapter answers

dataframe expr $\longrightarrow$ logical plan $\longrightarrow$ task groups $\longrightarrow$ successful attempt $\longrightarrow$ published sink

1. Which calls describe a plan, and which call demands execution?
2. Which producer files make one destination partition complete?
3. Which resources admit a task to an executor?
4. Which retry or speculative attempt supplies the consumed dataset?
5. Which guarantees belong to smallpond, and which come from storage?

Five questions the chapter answers

dataframe expr → logical plan → task groups → successful attempt → published sink

1. Which calls describe a plan, and which call demands execution?
2. Which producer files make one destination partition complete?
3. Which resources admit a task to an executor?
4. Which retry or speculative attempt supplies the consumed dataset?
5. Which guarantees belong to smallpond, and which come from storage?

Every later section answers exactly one of these five questions, in order.

Local notation for the ownership chain

Symbol	Meaning
$\mathcal{G} = (V, E)$	Planned task graph: task vertices, dependency edges

Local notation for the ownership chain

Symbol	Meaning
$\mathcal{G} = (V, E)$	Planned task graph: task vertices, dependency edges
$\mathcal{D}_v$	Dataset attached to the successful runtime of task v

Local notation for the ownership chain

Symbol	Meaning
$\mathcal{G} = (V, E)$	Planned task graph: task vertices, dependency edges
$\mathcal{D}_v$	Dataset attached to the successful runtime of task v
a_v	Successful attempt identity that owns $\mathcal{D}_v$

Local notation for the ownership chain

Symbol	Meaning
$\mathcal{G} = (V, E)$	Planned task graph: task vertices, dependency edges
$\mathcal{D}_v$	Dataset attached to the successful runtime of task v
a_v	Successful attempt identity that owns $\mathcal{D}_v$
$\mathcal{D}_{\text{pub}}$	Dependency-complete dataset exposed by the final sink

Local notation for the ownership chain

Symbol	Meaning
$\mathcal{G} = (V, E)$	Planned task graph: task vertices, dependency edges
$\mathcal{D}_v$	Dataset attached to the successful runtime of task v
a_v	Successful attempt identity that owns $\mathcal{D}_v$
$\mathcal{D}_{\text{pub}}$	Dependency-complete dataset exposed by the final sink

Definition (Published dependency dataset)

The file-backed output attached to the successful runtime of a completed task. A consumer follows that attached dataset through its dependency edge; it does not infer success from plausible files beneath a shared work directory.

System boundary and dataset contract

Two contracts, one usable dataset

Definition (Responsibility split)

smallpond owns lazy dataframe operators, their lowering to dependency-aware tasks, CPU/GPU/memory requests, and publication of file-backed datasets. It delegates namespace and file-operation semantics to the configured data root — 3FS, another shared mount, or a local filesystem.

Two contracts, one usable dataset

Definition (Responsibility split)

smallpond owns lazy dataframe operators, their lowering to dependency-aware tasks, CPU/GPU/memory requests, and publication of file-backed datasets. It delegates namespace and file-operation semantics to the configured data root — 3FS, another shared mount, or a local filesystem.

smallpond lane: logical plan and dependencies → successful task runtime → attached dataset $D(t)$

storage lane: path and file operations → backend placement and replication → durability and failure semantics

Two contracts, one usable dataset

Definition (Responsibility split)

smallpond owns lazy dataframe operators, their lowering to dependency-aware tasks, CPU/GPU/memory requests, and publication of file-backed datasets. It delegates namespace and file-operation semantics to the configured data root — 3FS, another shared mount, or a local filesystem.

smallpond lane: logical plan and dependencies → successful task runtime → attached dataset $D(t)$

storage lane: path and file operations → backend placement and replication → durability and failure semantics

The two lanes are drawn at the same three checkpoints on purpose — each smallpond claim has a storage claim directly beneath it.

The rule the scheduler cannot relax

For a dependency edge $u \rightarrow v$,

$\text{ready}(t_v) \implies \forall t_u \in \text{deps}(t_v), D(t_u) \text{ is attached to a successful runtime.}$

Invariant (Dependency-dataset selection)

Every downstream task consumes the dataset attached to the successful runtime of each declared dependency. A retry, scheduler restart, or speculative copy must not cause the consumer to combine attempt directories or select output by directory discovery.

The rule the scheduler cannot relax

For a dependency edge $u \rightarrow v$,

$$\text{ready}(t_v) \implies \forall t_u \in \text{deps}(t_v), D(t_u) \text{ is attached to a successful runtime.}$$

Invariant (Dependency-dataset selection)

Every downstream task consumes the dataset attached to the successful runtime of each declared dependency. A retry, scheduler restart, or speculative copy must not cause the consumer to combine attempt directories or select output by directory discovery.

This covers dataset choice only. A Python function's external side effects — an email, a database write — need their own idempotence policy; the invariant does not supply one.

Logical plans, repartition, and shuffle

The public API builds a LogicalPlan, not a result

Public call	Plan node	Contract before execution
<code>read_parquet,</code> <code>read_csv</code>	<code>DataSourceNode</code>	Name a dataset; no distributed read yet

The public API builds a LogicalPlan, not a result

Public call	Plan node	Contract before execution
<code>read_parquet,</code> <code>read_csv</code>	<code>DataSourceNode</code>	Name a dataset; no distributed read yet
<code>partial_sql</code>	<code>SqlEngineNode</code>	Runs per matched input partition, not as one global table

The public API builds a LogicalPlan, not a result

Public call	Plan node	Contract before execution
<code>read_parquet,</code> <code>read_csv</code>	<code>DataSourceNode</code>	Name a dataset; no distributed read yet
<code>partial_sql</code>	<code>SqlEngineNode</code>	Runs per matched input partition, not as one global table
<code>repartition,</code> <code>random_shuffle</code>	<code>HashPartitionNode</code> / <code>ShuffleNode</code>	Declare partition count and distribution before files move

The public API builds a LogicalPlan, not a result

Public call	Plan node	Contract before execution
<code>read_parquet,</code> <code>read_csv</code>	<code>DataSourceNode</code>	Name a dataset; no distributed read yet
<code>partial_sql</code>	<code>SqlEngineNode</code>	Runs per matched input partition, not as one global table
<code>repartition,</code> <code>random_shuffle</code>	<code>HashPartitionNode</code> / <code>ShuffleNode</code>	Declare partition count and distribution before files move
<code>write_parquet</code>	<code>DataSinkNode</code>	Triggers planning and execution; a materialization boundary

The public API builds a LogicalPlan, not a result

Public call	Plan node	Contract before execution
<code>read_parquet,</code> <code>read_csv</code>	<code>DataSourceNode</code>	Name a dataset; no distributed read yet
<code>partial_sql</code>	<code>SqlEngineNode</code>	Runs per matched input partition, not as one global table
<code>repartition,</code> <code>random_shuffle</code>	<code>HashPartitionNode</code> / <code>ShuffleNode</code>	Declare partition count and distribution before files move
<code>write_parquet</code>	<code>DataSinkNode</code>	Triggers planning and execution; a materialization boundary

An action is a materialization boundary — it does not retroactively make earlier calls eager.

A hash repartition is a many-to-many file exchange

For q destinations and an internal producer count r ($r \leq 100$, $rq \leq 4,096,000$), producer i writes $P_{i,j}$ for every destination j , and $\mathcal{I}_j = \{P_{0,j}, P_{1,j}, \dots, P_{r-1,j}\}$.

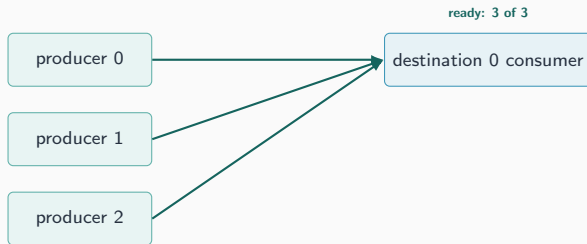
producer 0

producer 1

producer 2

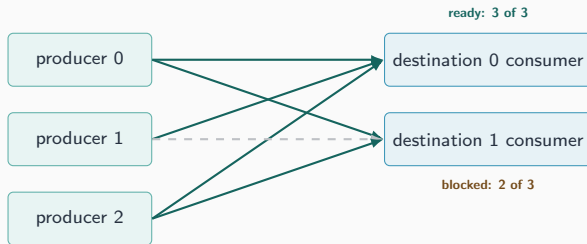
A hash repartition is a many-to-many file exchange

For q destinations and an internal producer count r ($r \leq 100$, $rq \leq 4,096,000$), producer i writes $P_{i,j}$ for every destination j , and $\mathcal{I}_j = \{P_{0,j}, P_{1,j}, \dots, P_{r-1,j}\}$.



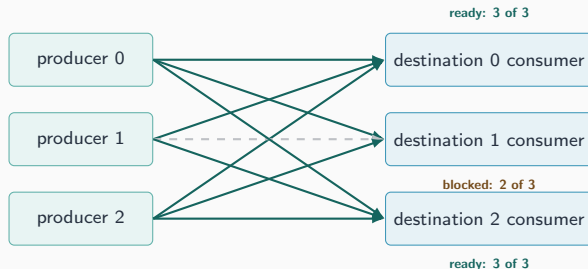
A hash repartition is a many-to-many file exchange

For q destinations and an internal producer count r ($r \leq 100$, $r q \leq 4,096,000$), producer i writes $P_{i,j}$ for every destination j , and $\mathcal{I}_j = \{P_{0,j}, P_{1,j}, \dots, P_{r-1,j}\}$.



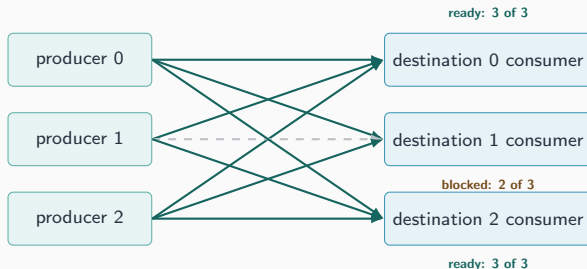
A hash repartition is a many-to-many file exchange

For q destinations and an internal producer count r ($r \leq 100$, $rq \leq 4,096,000$), producer i writes $P_{i,j}$ for every destination j , and $\mathcal{I}_j = \{P_{0,j}, P_{1,j}, \dots, P_{r-1,j}\}$.



A hash repartition is a many-to-many file exchange

For q destinations and an internal producer count r ($r \leq 100$, $rq \leq 4,096,000$), producer i writes $P_{i,j}$ for every destination j , and $\mathcal{I}_j = \{P_{0,j}, P_{1,j}, \dots, P_{r-1,j}\}$.



Three destinations wait on the corresponding output from every one of the r producers — not on “enough files look right.”

Execution, resource admission, and failure

Two paths, one LogicalPlan

Path	Behavior
Interactive Session	Optimizes the lazy plan, memoizes node-to-task mapping, submits through Ray on action

Two paths, one LogicalPlan

Path	Behavior
Interactive Session	Optimizes the lazy plan, memoizes node-to-task mapping, submits through Ray on action
Batch JobManager	Atomically persists runtime context, logical plan, and execution plan before the custom scheduler starts

Two paths, one LogicalPlan

Path	Behavior
Interactive Session	Optimizes the lazy plan, memoizes node-to-task mapping, submits through Ray on action
Batch JobManager	Atomically persists runtime context, logical plan, and execution plan before the custom scheduler starts

- A restart can reload dumped scheduler state and increment `sched_epoch`.

Two paths, one LogicalPlan

Path	Behavior
Interactive Session	Optimizes the lazy plan, memoizes node-to-task mapping, submits through Ray on action
Batch JobManager	Atomically persists runtime context, logical plan, and execution plan before the custom scheduler starts

- A restart can reload dumped scheduler state and increment `sched_epoch`.
- Ray or the platform launcher changes *how* planned work reaches workers — not the DuckDB, Arrow, or partition semantics inside a task.

Resource requests, not automatic isolation

Definition (Resource-admission record)

Distinguishes the CPU, GPU, and memory quantities requested by a task; the free quantities reported by the selected executor; any overcommit ratio or retry-time boost; and the mechanism, if any, that enforces a process limit.

- Every logical Node carries `cpu_limit`, `gpu_limit`, `memory_limit` — the base node documentation says these are **not enforced**.

Resource requests, not automatic isolation

Definition (Resource-admission record)

Distinguishes the CPU, GPU, and memory quantities requested by a task; the free quantities reported by the selected executor; any overcommit ratio or retry-time boost; and the mechanism, if any, that enforces a process limit.

- Every logical Node carries `cpu_limit`, `gpu_limit`, `memory_limit` — the base node documentation says these are **not enforced**.
- DuckDB tasks apply their own CPU/memory overcommit ratios; an out-of-memory failure can boost the next request.

Resource requests, not automatic isolation

Definition (Resource-admission record)

Distinguishes the CPU, GPU, and memory quantities requested by a task; the free quantities reported by the selected executor; any overcommit ratio or retry-time boost; and the mechanism, if any, that enforces a process limit.

- Every logical Node carries `cpu_limit`, `gpu_limit`, `memory_limit` — the base node documentation says these are **not enforced**.
- DuckDB tasks apply their own CPU/memory overcommit ratios; an out-of-memory failure can boost the next request.

Admission is evidence the scheduler found reported capacity — not proof execution stayed within the request.

Retry, speculation, and first success

Invariant (Attempt publication)

Retries, scheduler epochs, and speculative copies retain distinct attempt and staging identities. Only the first successful runtime attaches the dependency dataset consumed downstream; redundant or failed attempt directories never become additional inputs.

- An ordinary exception increments `fail_count` and retries until a configured ceiling.

Retry, speculation, and first success

Invariant (Attempt publication)

Retries, scheduler epochs, and speculative copies retain distinct attempt and staging identities. Only the first successful runtime attaches the dependency dataset consumed downstream; redundant or failed attempt directories never become additional inputs.

- An ordinary exception increments `fail_count` and retries until a configured ceiling.
- After 20 completed peers, a straggler crossing a p95/p99-derived threshold gets a speculative duplicate; data sinks opt out.

Retry, speculation, and first success

Invariant (Attempt publication)

Retries, scheduler epochs, and speculative copies retain distinct attempt and staging identities. Only the first successful runtime attaches the dependency dataset consumed downstream; redundant or failed attempt directories never become additional inputs.

- An ordinary exception increments `fail_count` and retries until a configured ceiling.
- After 20 completed peers, a straggler crossing a p95/p99-derived threshold gets a speculative duplicate; data sinks opt out.

Selecting one attached dataset does not undo a losing attempt's external side effect.

Storage backend, 3FS boundary, and GraySort evidence

Same logical plan, three systems meanings

Data root	What changes
3FS mount	Namespace, chunk placement, replication, client, failure behavior of the 3FS chapter

Same logical plan, three systems meanings

Data root	What changes
3FS mount	Namespace, chunk placement, replication, client, failure behavior of the 3FS chapter
Other shared mount	Consistency, visibility, link behavior, recovery follow that filesystem

Same logical plan, three systems meanings

Data root	What changes
3FS mount	Namespace, chunk placement, replication, client, failure behavior of the 3FS chapter
Other shared mount	Consistency, visibility, link behavior, recovery follow that filesystem
Local filesystem	Paths are not automatically visible to other hosts; placement is a platform concern

Same logical plan, three systems meanings

Data root	What changes
3FS mount	Namespace, chunk placement, replication, client, failure behavior of the 3FS chapter
Other shared mount	Consistency, visibility, link behavior, recovery follow that filesystem
Local filesystem	Paths are not automatically visible to other hosts; placement is a platform concern

smallpond's HF3FS-path detection changes whether a sink links or copies — it does not move filesystem protocol ownership into the dataframe layer.

The GraySort row is source-reported, not reproduced here

[REPORTED: smallpond README / GraySort benchmark] The public deployment processed 110.5 TiB in 30 minutes 14 seconds on 50 compute and 25 3FS storage nodes.

- Its validation flag (`--validate_results`) is `false by default` — “the process exited” and “the sort passed an independent validity check” are separate observations.

The GraySort row is source-reported, not reproduced here

[REPORTED: smallpond README / GraySort benchmark] The public deployment processed 110.5 TiB in 30 minutes 14 seconds on 50 compute and 25 3FS storage nodes.

- Its validation flag (`--validate_results`) is [false by default](#) — “the process exited” and “the sort passed an independent validity check” are separate observations.
- This project has no matching smallpond/3FS deployment: GraySort reproduction is [administratively closed](#), not a measured zero or failure.

From concept to code: a rename makes a dataset attached

The definition “published dependency dataset” has a literal home: a successful attempt’s staging directory becomes the final path by rename.

```
from smallpond/execution/task.py

def finalize(self):
    assert self.status == WorkStatus.SUCCEED
    if self.runtime_output_abspath != self.final_output_abspath \
        and os.path.exists(self.runtime_output_abspath):
        os.rename(self.runtime_output_abspath, self.final_output_abspath)
```


From concept to code: a rename makes a dataset attached

The definition “published dependency dataset” has a literal home: a successful attempt’s staging directory becomes the final path by rename.

```
from smallpond/execution/task.py

def finalize(self):
    assert self.status == WorkStatus.SUCCEED
    if self.runtime_output_abspath != self.final_output_abspath \
        and os.path.exists(self.runtime_output_abspath):
        os.rename(self.runtime_output_abspath, self.final_output_abspath)
```

Before this rename, the attempt’s files sit under a runtime-id staging path. After it, exactly one path exists at the final location — which is what lets a downstream consumer follow a dependency edge instead of scanning a directory. [INSPECTED: smallpond execution/task.py]

Concept-to-code map for the chapter

- dataframe facade: lazy calls to logical nodes → `smallpond/smallpond/dataframe.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`
- **attempt identity, staging, finalize/publication** → `smallpond/execution/task.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`
- **attempt identity, staging, finalize/publication** → `smallpond/execution/task.py`
- **batch persistence and scheduler-epoch reload** → `smallpond/execution/manager.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`
- **attempt identity, staging, finalize/publication** → `smallpond/execution/task.py`
- **batch persistence and scheduler-epoch reload** → `smallpond/execution/manager.py`
- **data_root resolution** → `smallpond/smallpond/session.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`
- **attempt identity, staging, finalize/publication** → `smallpond/execution/task.py`
- **batch persistence and scheduler-epoch reload** → `smallpond/execution/manager.py`
- **data_root resolution** → `smallpond/smallpond/session.py`
- **HF3FS mount detection** → `smallpond/io/filesystem.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`
- **attempt identity, staging, finalize/publication** → `smallpond/execution/task.py`
- **batch persistence and scheduler-epoch reload** → `smallpond/execution/manager.py`
- **data_root resolution** → `smallpond/smallpond/session.py`
- **HF3FS mount detection** → `smallpond/io/filesystem.py`
- **public GraySort benchmark script** → `smallpond/benchmarks/gray_sort_benchmark.py`

Concept-to-code map for the chapter

- **dataframe facade: lazy calls to logical nodes** → `smallpond/smallpond/dataframe.py`
- **logical node and dataset identity** → `smallpond/logical/node.py`,
`smallpond/logical/dataset.py`
- **node-to-task-group lowering, shuffle expansion** → `smallpond/logical/planner.py`
- **admission, retry, speculation, first success** → `smallpond/execution/scheduler.py`
- **attempt identity, staging, finalize/publication** → `smallpond/execution/task.py`
- **batch persistence and scheduler-epoch reload** → `smallpond/execution/manager.py`
- **data_root resolution** → `smallpond/smallpond/session.py`
- **HF3FS mount detection** → `smallpond/io/filesystem.py`
- **public GraySort benchmark script** → `smallpond/benchmarks/gray_sort_benchmark.py`

A plan describes work. Only an attached, successful dataset can be trusted.

Next: the precision and numerical-correctness chapter shows a successfully published artifact can still be wrong (Chapter 32).